



Made for humans,
powered by AI

Inteligência Artificial

Treinamento



Vitória Firmino
Product Owner

- Parte 1: entender como LLMs funcionam e quais são seus limites reais.
- Parte 2: decidir como usar LLMs em sistemas reais, com qualidade, custo, segurança e responsabilidade.



Fundamentos



Conceitos Importantes

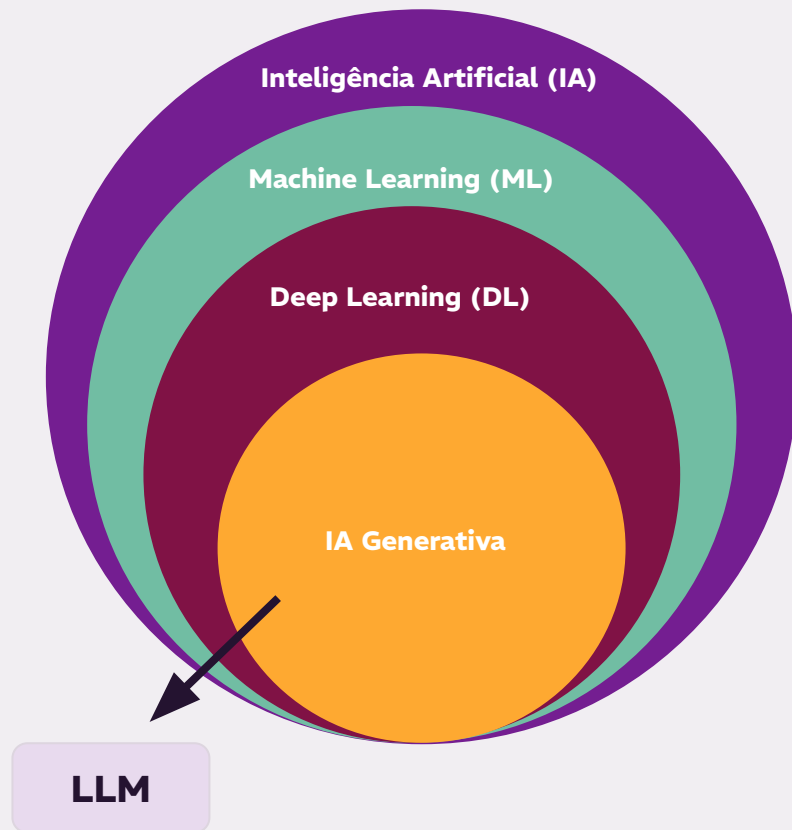
Inteligência Artificial: Área que desenvolve sistemas capazes de executar tarefas que normalmente exigem inteligência humana.

Machine Learning: Subcampo da IA em que modelos aprendem padrões a partir de dados para realizar previsões ou decisões.

Deep Learning: Abordagem de ML baseada em redes neurais profundas que aprendem representações complexas dos dados em múltiplas camadas.

IA Generativa: Tipo de IA capaz de criar novos conteúdos a partir de padrões aprendidos de **grandes volumes de dados**.

LLM (Large Language Model): Modelo de IA treinado em enormes conjuntos de dados para compreender e gerar linguagem natural.



Machine Learning

Supervisionado

O modelo aprende a partir de dados rotulados (com respostas conhecidas).

O que o modelo recebe:

Entrada X + saída esperada Y

Não Supervisionado

O modelo explora padrões sem saber a resposta certa.

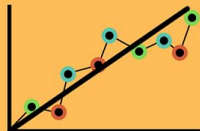
O que o modelo recebe:

Apenas os dados de Entrada X

Classificação



Regressão



Agrupamento



Classificação: Identificar se um e-mail é *spam* ou *não spam*.

Regressão: Prever o preço de uma casa com base em área, localização e número de quartos.

Agrupamento: Agrupar clientes com comportamentos de compra semelhantes sem rótulos prévios.

Comprimento da sépala	Largura da sépala	Comprimento da pétala	Largura da pétala	Espécie
5.1	3.5	1.4	0.2	Setosa
4.9	3	1.4	0.2	Setosa
4.7	3.2	1.3	0.2	Setosa
4.6	3.1	1.5	0.2	Setosa
5	3.6	1.4	0.2	Setosa
5.4	3.9	1.7	0.4	Setosa
4.6	3.4	1.4	0.3	Setosa
5	3.4	1.5	0.2	Setosa
4.4	2.9	1.4	0.2	Setosa
4.9	3.1	1.5	0.1	Setosa
5.4	3.7	1.5	0.2	Setosa

Comprimento da sépala	Largura da sépala	Comprimento da pétala	Largura da pétala
5.1	3.5	1.4	0.2
4.9	3	1.4	0.2
4.7	3.2	1.3	0.2
4.6	3.1	1.5	0.2
5	3.6	1.4	0.2
5.4	3.9	1.7	0.4
4.6	3.4	1.4	0.3
5	3.4	1.5	0.2
4.4	2.9	1.4	0.2
4.9	3.1	1.5	0.1
5.4	3.7	1.5	0.2

DADOS ESTRUTURADOS

ID	NOME	IDADE
1	John	35
2	Mary	28

CARACTERÍSTICAS:

- organizados
- formato definido

EXEMPLOS:



DADOS NÃO ESTRUTURADOS



DESORGANIZADOS

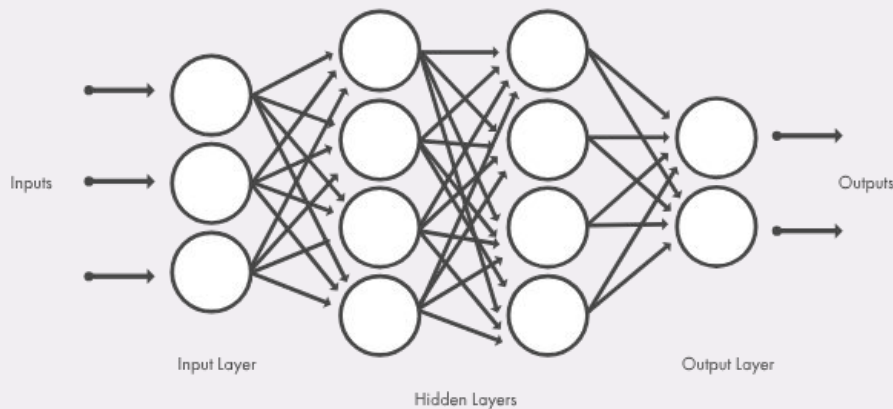
- sem formato pré-definido

EXEMPLOS



Deep Learning

O deep learning resolve o problema do machine learning tradicional de depender de muita intervenção humana para extrair boas características dos dados.



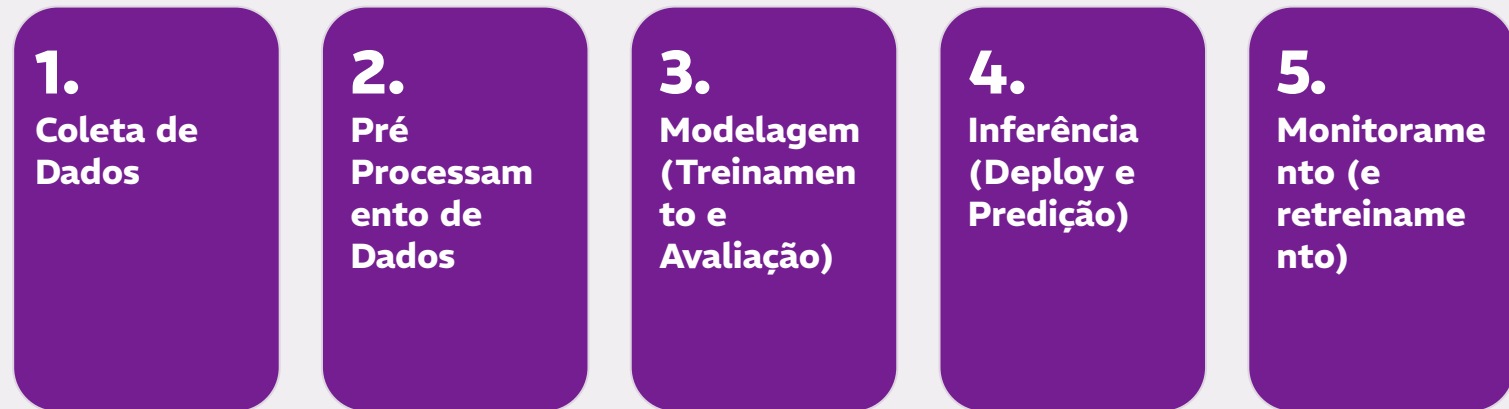
A própria rede aprende sozinha quais características são importantes mesmo quando os dados são muito complexos, como imagens, áudio e texto.

Como a IA aprende?



A IA aprende com dados e feedback, não por "inteligência" humana. → Probabilístico

Pipeline de IA



É essencial garantir qualidade, relevância e confiabilidade dos dados, além de assegurar privacidade e conformidade com a LGPD. O pré-processamento é a etapa mais trabalhosa e, quando mal conduzido, pode introduzir ou amplificar vieses, afetando diretamente a performance do modelo.

Métricas de Avaliação - ML Clássico

Acurácia: porcentagem total de previsões corretas.

Classificação de imagens de roupas em um e-commerce.

Se errar uma peça ou outra, o impacto é pequeno, e todas as categorias têm quantidades parecidas.

quando erros custam pouco e as classes são balanceadas

Precisão: entre os itens que o modelo disse ser positivos, quantos realmente eram positivos.

Detectar fraude em cartão de crédito.

Se o modelo marcar transações legítimas como fraude (falsos positivos), usuários ficam bloqueados à toa.

quando falsos positivos são perigosos

Recall: entre os positivos reais, quantos o modelo conseguiu encontrar.

Diagnóstico de câncer em exames médicos.

O pior erro é deixar um caso real de câncer passar despercebido (falso negativo).

quando falsos negativos são perigosos

F1-score: média harmônica entre precisão e recall, equilibrando os dois em um único número.

Moderação automática de discursos de ódio.

O modelo precisa identificar agressões (recall), mas sem censurar conteúdo legítimo (precisão).

quando é preciso equilibrar precisão e recall

Conclusões

1. Escolha do modelo não é aleatória: depende diretamente do tipo de dado e da tarefa;
2. A qualidade do experimento depende da divisão correta dos dados: treino, validação e teste evitam overfitting e dão previsibilidade em produção.
3. Modelos aprendem com dados, não com magia: o treinamento é apenas repetição em larga escala, refinando pesos até reconhecer padrões.
4. Iteração é inevitável: não existe “modelo perfeito de primeira”. PO precisa prever ciclos e tempo de experimentação.
5. PO deve garantir critérios claros: o time precisa saber quando um modelo está “bom o suficiente” para avançar no pipeline.
6. Avaliação precisa ser contínua: métricas diferentes respondem a objetivos distintos (ex: recall em saúde, precisão em fraude).

Como LLMs Funcionam



Pré-treinamento

- Aprende linguagem em larga escala
- Textos da internet, livros, códigos

O pré-treinamento é quando o modelo aprende a *linguagem em si*. Ele não aprende regras de negócio, nem contexto da sua empresa.

Janela de contexto

A **janela de contexto** funciona como a *memória de trabalho* do LLM.

Ela define:

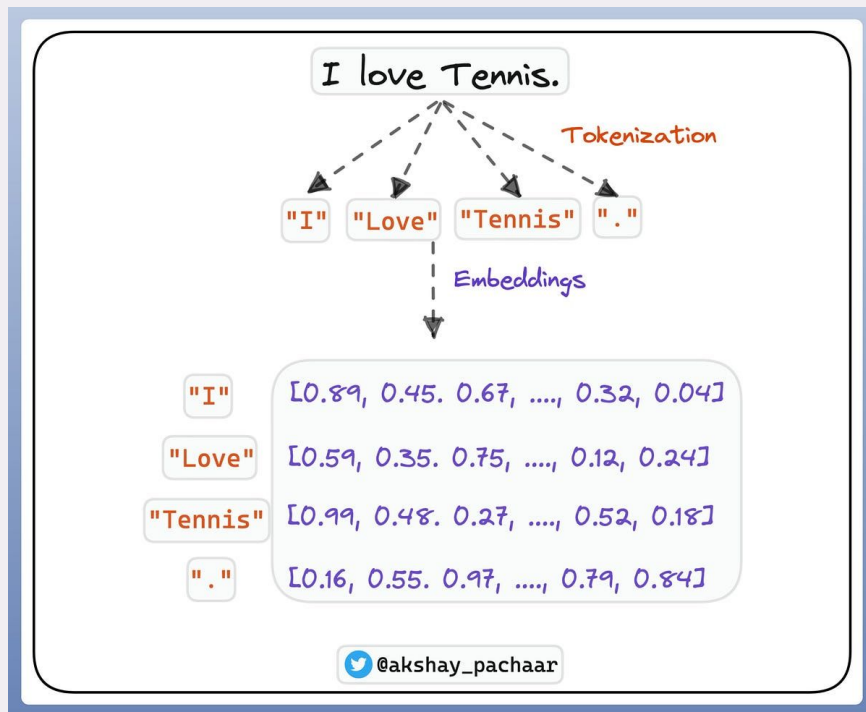
- Quanto de uma conversa o modelo consegue manter
- O tamanho máximo de documentos ou códigos processados de uma vez

Quando o conteúdo excede essa janela:

- Parte do contexto é **truncada**
- Ou precisa ser **resumida** para continuar a interação

Tokens e Embeddings

- Texto é quebrado em **tokens**
- Tokens são transformados em **embeddings** (**vetores numéricos**)
- O modelo opera sobre **vetores**, não sobre palavras
- A **janela de contexto** limita o que o modelo consegue usar



Atenção

Atenção é o mecanismo que permite ao LLM:

- Pesquisar quais partes do contexto são mais relevantes
- Relacionar tokens distantes no texto
- Focar no que importa para gerar a próxima resposta

Nem todo token recebe o mesmo peso. Dependendo da pergunta, o modelo “olha” mais para uma palavra específica.

Frase: “O cliente reclamou do atraso na entrega, mas elogiou o atendimento.”

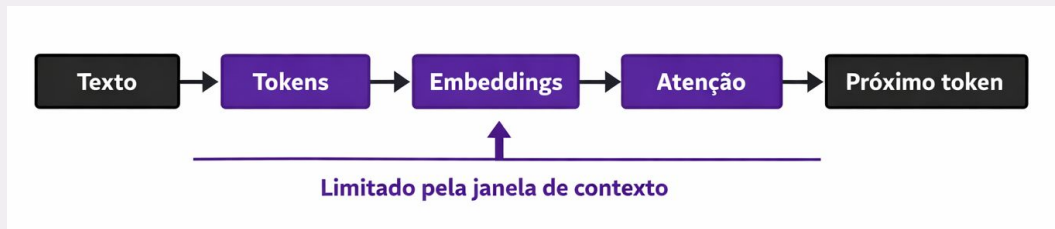
Pergunta 1: O que o cliente achou do atendimento?

Frase: “O cliente reclamou do atraso na entrega, mas elogiou o **atendimento**.” → O peso de atendimento vai ser maior

Pergunta 2: Houve alguma reclamação?

Frase: “O cliente reclamou do **atraso** na entrega, mas elogiou o atendimento.” → O peso de atraso vai ser maior

O caminho que um texto percorre



O processo começa com o **texto** de entrada, que é dividido em **tokens** (palavras ou partes delas). Cada token é convertido em um **embedding**, uma representação numérica que permite ao modelo trabalhar com significado. O mecanismo de **atenção** avalia quais partes do texto são mais relevantes e, com base nisso, o modelo gera o **próximo token** de forma probabilística, construindo a resposta passo a passo. Todo esse processo é limitado pela **janela de contexto**, que define quanto do texto o modelo consegue considerar ao mesmo tempo. Quando esse limite é excedido, partes do conteúdo deixam de ser consideradas.

Modelo Base

Um **modelo base** é o ponto de partida. Ele é treinado em larga escala para prever o próximo token a partir de enormes volumes de texto bruto, como livros, artigos e páginas da web. O objetivo aqui é aprender a estrutura da língua, padrões sintáticos, semânticos e conhecimento geral. Ele sabe “como a linguagem funciona”, mas **não sabe necessariamente obedecer instruções nem interagir com humanos de forma adequada.**

Especialização: Instruction-tuned

Já um **modelo ajustado para receber instruções** é um modelo base que passou por uma etapa adicional de treinamento supervisionado com **pares de instrução → resposta**. Nessa fase, o modelo aprende explicitamente que, quando recebe uma instrução ("resuma", "explique", "classifique", "gere"), ele deve seguir a tarefa solicitada. Isso muda bastante o comportamento: o modelo se torna mais útil para tarefas diretas, mais previsível e mais alinhado ao formato esperado. Ainda assim, ele não é otimizado para diálogo contínuo — cada prompt é tratado como uma tarefa isolada.

Especialização: Chat Models

Os **chat models** são uma evolução dos instruction-tuned, pensados especificamente para **conversação multi-turno**. Além do treinamento com instruções, eles passam por etapas onde aprendem a manter contexto, respeitar tom, evitar respostas perigosas, reconhecer quando não sabem algo e interagir de forma mais natural. Eles entendem papéis ("usuário", "assistente"), lidam melhor com follow-ups e ajustam a resposta com base no histórico da conversa. Em essência, não são apenas bons em cumprir tarefas, mas em **conversar como um assistente**.

Inferência: por que cada resposta pode variar?

Inferência é o momento em que o modelo usa o que aprendeu para gerar uma resposta. Toda vez que você envia um prompt, o modelo:

1. Lê o contexto e calcula probabilidades para o próximo token
2. Escolhe um token
3. Repete o processo

Amostragem: Em vez de **sempre** pegar o topo, o modelo amostra dessa distribuição.

Isso significa:

- respostas podem variar
- estrutura muda
- exemplos mudam
- palavras mudam

Mesmo com o mesmo prompt. Essa aleatoriedade controlada é o que torna o modelo natural e criativo.

Top-k

O modelo não precisa considerar tudo.

Top-k: só os k tokens mais prováveis

Ex: Top 10.

Eu gosto de comer pizza .	}	Top 10
Eu gosto de comer pizza .		
Eu gosto de comer pizza .		
Eu gosto de comer pizza .		
Eu gosto de comer maçãs .		
Eu gosto de comer maçãs .		
Eu gosto de comer pizza .		
Eu gosto de comer pizza .		
Eu gosto de comer pizza .		
Eu gosto de comer pizza .		

Temperatura

A temperatura controla a distribuição:

Temperatura baixa (0.1–0.3)

→ respostas mais previsíveis

→ ideal para classificação, QA factual

Temperatura média (0.6–0.8)

→ equilíbrio

→ ideal para chat, explicação

Temperatura alta (1.0+)

→ mais variação

→ mais criatividade

→ mais risco de erro

Temperatura 0	Temperatura 0.7	Temperatura 1
Eu gosto de comer pizza .	Eu gosto de comer pizza .	Eu gosto de comer pizza .
Eu gosto de comer pizza .	Eu gosto de comer pizza .	Eu gosto de comer sushi .
Eu gosto de comer pizza .	Eu gosto de comer pizza .	Eu gosto de comer lasanha .
Eu gosto de comer pizza .	Eu gosto de comer maçãs .	Eu gosto de comer frutas .
Eu gosto de comer maçãs .	Eu gosto de comer maçã .	Eu gosto de comer chocolate .
Eu gosto de comer maçãs .	Eu gosto de comer lasanha .	Eu gosto de comer frutas .
Eu gosto de comer pizza .	Eu gosto de comer pizza .	Eu gosto de comer maçãs .
Eu gosto de comer pizza .	Eu gosto de comer pizza .	Eu gosto de comer maçãs .
Eu gosto de comer pizza .	Eu gosto de comer pizza .	Eu gosto de comer pizza .
Eu gosto de comer pizza .	Eu gosto de comer maçãs .	Eu gosto de comer pizza .

Top-p

O modelo escolhe entre as palavras mais prováveis cuja soma de probabilidades atinge um valor P que definimos (ex.: 0.9). Isso limita o vocabulário às opções mais relevantes, mantendo diversidade controlada.

Vamos supor que o modelo está tentando completar a frase: "O céu hoje está...", e ele gera estas possíveis próximas palavras com suas probabilidades:

- azul – 0.4
- nublado – 0.3
- limpo – 0.15
- chuvoso – 0.1
- neon – 0.03
- radioativo – 0.02

O modelo soma as palavras em ordem de maior probabilidade até chegar a 90%:

- azul (0.4)
- nublado (0.3) → soma = 0.7
- limpo (0.15) → soma = 0.85
- chuvoso (0.1) → soma = 0.95 → passou de 0.9

Nesse caso, ele considera apenas "azul", "nublado" e "limpo" como opções para amostragem. As outras (mais improváveis) são descartadas.

Contexto muda tudo

Pequenas mudanças no contexto:

- histórico da conversa
- ordem das mensagens
- system prompt
- exemplos anteriores

Mudam completamente:

- a distribuição de probabilidade
- o caminho de geração

Por isso:

Dois chats “iguais” nunca são 100% iguais.

Alucinação

Alucinação acontece quando o modelo gera uma resposta linguisticamente plausível, mas factualmente incorreta ou não verificável, sem sinalizar incerteza.

Motivo 1 — O objetivo do modelo não é verdade

LLMs são treinados para prever o próximo token mais provável, não para verificar fatos no mundo real.

Motivo 2 — Ausência de conhecimento ≠ ausência de resposta e Falta de grounding externo

Quando o modelo não sabe, não tem contexto suficiente, ou foi mal instruído, ele não fica em silêncio por padrão. Ele tenta “gerar a melhor resposta possível”. Isso pode resultar em respostas inventadas, mas bem escritas. Quando não há RAG, tools ou fontes explícitas o modelo depende apenas do que está nos pesos do modelo.

Motivo 3 — Pressão do prompt

Prompts como: “explique detalhadamente”, “liste todos” ou “cite exemplos” empurram o modelo a preencher lacunas, mesmo quando a informação não existe.

Limitações

Limitação 1 — Conhecimento estático

O modelo não aprende durante a conversa, não atualiza fatos e não acessa a internet (por padrão). Ele reflete o estado do mundo no momento do treino.

Limitação 2 — Janela de contexto

O modelo só “enxerga” até N tokens esquece o que sai da janela, não tem memória persistente real. Isso afeta chats longos e documentos grandes e histórico acumulado.

Limitação 3 — Não há verificação interna de verdade

O modelo não valida fatos, não cruza fontes e não sabe se algo é verdadeiro. Ele sabe o que parece correto linguisticamente.

Limitações

Limitação 4 — Raciocínio é sequencial, não simbólico

Mesmo quando “parece raciocinar”, não executa lógica formal, não mantém estados explícitos e não garante consistência global. Por isso, pode errar contas simples, pode se contradizer e pode “mudar de ideia” no meio da resposta.

Limitação 5 — Sensível à forma do prompt

Mudanças pequenas como: ordem das frases, exemplos/dados ou tom mudam o caminho de inferência e a resposta final. Isso dificulta reprodutibilidade, auditoria e explicabilidade forte.

Limitação 6 — Não entende impacto social por si só

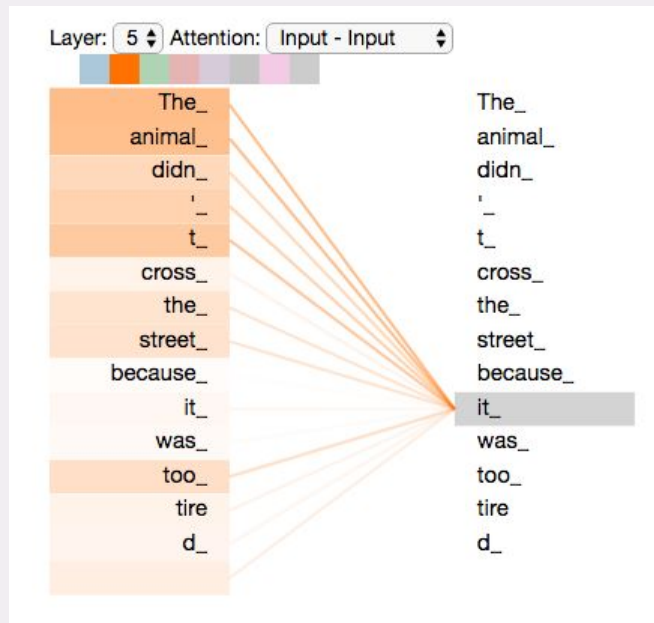
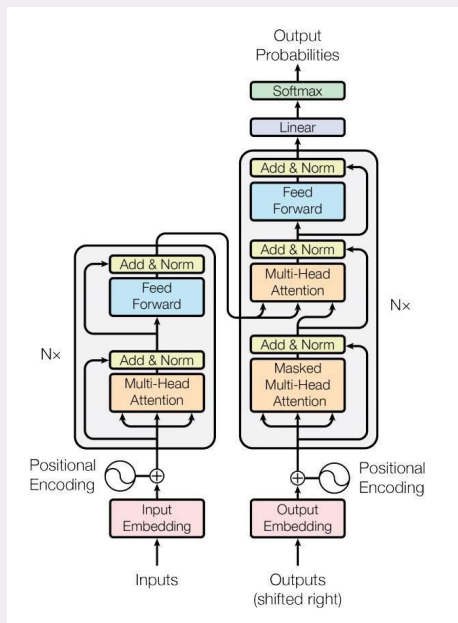
O modelo replica padrões dos dados, herda vieses e não tem noção ética própria. Não tem regras, métricas e avaliação humana. Ele não sabe o que é justo ou injusto.

Transformers



Transformers

Um tipo avançado de arquitetura de rede neural, lançada em 2017. Antes dele, os modelos processavam o texto palavra por palavra. O Transformer mudou tudo ao permitir que o modelo olhe para todas as palavras ao mesmo tempo, usando mecanismos de **atenção**.



“O banco quebrou”
Sem contexto, “banco” é ambíguo.
Com atenção, o modelo aprende a relacionar “banco” com “quebrou” e o restante da frase.

Encoder-Decoder

Como funciona:

1. Encoder lê todo o texto de entrada de uma vez e cria uma representação rica.
2. Decoder usa essa representação para gerar a saída, token por token.

É literalmente:

“Entendi o texto → agora vou gerar algo a partir disso”.

Quando isso é ótimo

- Tradução
- Resumo
- Parafraseamento
- Geração condicionada

Família de modelos:

- T5
- BART
- mT5

Encoder-only

Como funciona:

1. O modelo lê toda a frase ao mesmo tempo.
2. Cada palavra pode olhar para todas as outras (atenção bidirecional).
3. O resultado é uma representação semântica do texto.

O que ele não faz:

1. Não gera texto.
2. Não escreve respostas.

Quando isso é ótimo

- Classificação
- Análise de sentimento
- Detecção de viés

Família de modelos:

- BERT
- BERTimbau
- RoBERTa
- DistilBERT

Decoder-only

Como funciona:

1. O modelo vê apenas o passado.
2. Ele aprende prevendo o próximo token.
3. Para gerar bem, ele precisa compreender profundamente o contexto.

A regra é:

entender → prever → gerar → repetir

Quando isso é ótimo

- Chat
- Geração livre
- Raciocínio em linguagem natural
- Agentes

Família de modelos:

- OpenAI GPT
- Meta LLaMA
- Mistral AI Mistral
- Qwen
- Falcon

Casos de Uso de LLMs em Produtos Digitais



Chatbots

Nesse modelo, o usuário conduz a interação. O LLM responde com base no contexto fornecido, podendo ou não estar conectado a uma base de conhecimento via RAG. A experiência é simples, mas o risco de alucinação cresce quando o domínio não está bem delimitado ou quando o chatbot tenta responder perguntas que exigiriam verificação externa.

Costuma ser usado para:

- atendimento ao cliente,
- esclarecimento de dúvidas frequentes,
- suporte técnico básico,
- acesso facilitado a informações.

Chatbots funcionam melhor quando:

- o escopo é restrito,
- há tolerância a respostas genéricas,
- o impacto de erro é baixo.

Copilotos

O foco do copiloto é **ajudar** a fazer. Ele atua como um parceiro cognitivo, oferecendo sugestões, rascunhos, recomendações contextuais, explicações sob demanda. Exemplos típicos incluem:

- copilotos de código
- copilotos de escrita
- copilotos de análise de dados

Diferente do chatbot, o copiloto:

- entende o estado atual do trabalho,
- se integra a ferramentas,
- opera dentro de um fluxo específico.

Aqui, a confiança é fundamental. O modelo precisa ser previsível, explicável e frequentemente combinado com validações automáticas ou humanas. Por isso, copilotos costumam usar prompts estruturados, ferramentas externas, regras claras de atuação e limites explícitos de autonomia.

Assistentes

Assistentes são o nível mais avançado de uso de LLMs em produtos digitais. Eles não apenas interagem, mas executam ações dentro do sistema, muitas vezes de forma semiautônoma. O foco do assistente é orquestrar, decidir e agir.

Um assistente:

- interpreta intenções,
- coordena múltiplas etapas,
- chama APIs e ferramentas,
- mantém estado e contexto,
- segue regras de negócio.

Nesse caso, o LLM deixa de ser apenas uma interface de linguagem e passa a ser parte da lógica operacional do produto. Isso exige guardrails fortes, validação de ações, controle de autonomia, monitoramento constante.

Agentes

Chatbot, copiloto e assistente são formas de interação. **Agentes** são entidades de decisão e execução por trás. Um agente é um componente que:

- tem um objetivo claro,
- recebe estado e contexto,
- decide o que fazer,
- pode executar ações (ou pedir confirmação),
- segue regras e limites.

Ele pode usar um LLM para raciocinar, ferramentas (APIs, banco, serviços), memória (curto ou longo prazo), validações.

Como isso aparece na Interface

Copiloto

UI/UX típica:

- painéis laterais
- sugestões inline
- botões “Aplicar sugestão”
- comentários contextuais

Exemplo:

ferramenta de análise com insights laterais

Assistente

UI/UX típica:

- dashboards
- cards de decisão
- timelines de ações
- notificações proativas
- logs de explicabilidade

Exemplo:

sistema que gera e executa planos

Resumindo...

- Chatbot → interface conversacional
- Copiloto → assistência embutida no fluxo
- Assistente → execução orientada a objetivo
- Agentes → cérebro operacional por trás de tudo

Classificação, Sumarização e Extração de Informação com LLMs

Essas três capacidades formam a base de muitos sistemas “invisíveis” que usam LLMs por trás da interface. Diferente de chatbots, aqui o LLM processa texto para gerar estrutura, não apenas linguagem fluida.

Classificação: é a tarefa de atribuir um rótulo a um texto com base em seu conteúdo.

Sumarização: é a tarefa de condensar um texto longo, preservando os pontos mais importantes.

Extração de Informações: é a tarefa de identificar e estruturar elementos específicos dentro de um texto livre.

LLMs não servem apenas para “conversar”. Eles são excelentes em ler, decidir, resumir e estruturar informação.

Como LLMs fazem isso: Classificação

O modelo:

1. lê o texto,
2. compreende o contexto,
3. decide qual rótulo é mais apropriado,
4. retorna uma resposta estruturada (classe, score, justificativa).

Isso pode ser feito: sem treino adicional (zero-shot), com exemplos no prompt (few-shot), ou com fine-tuning (quando disponível).

Onde isso aparece no produto

- triagem automática
- priorização de tarefas
- roteamento de tickets
- segmentação de usuários
- moderação de conteúdo

Risco principal

Classificação parece determinística, mas ainda é probabilística. Por isso, em contextos críticos, é comum exigir score de confiança, usar revisão humana e combinar com regras fixas.

Como LLMs fazem isso: Sumarização

O modelo:

1. identifica os temas centrais,
2. ignora detalhes redundantes,
3. reorganiza a informação,
4. gera uma versão mais curta e legível.

LLMs são especialmente bons nisso porque entendem contexto global, reconhecem relevância semântica e mantêm coerência textual.

Onde isso aparece no produto

- resumos de reuniões
- síntese de documentos
- dashboards explicativos
- relatórios automáticos
- briefing para tomada de decisão

Risco principal

Sumarização pode omitir detalhes críticos, suavizar riscos e mudar o tom original. Por isso, bons produtos deixam claro que é um resumo, permitem acesso ao texto original, explicam critérios de sumarização.

Como LLMs fazem isso: Extração de Informação

O modelo:

1. lê o texto,
2. reconhece padrões linguísticos e semânticos,
3. mapeia trechos para campos estruturados,
4. retorna JSON, tabelas ou pares chave–valor.

Aqui, o LLM funciona como um tradutor de linguagem natural para **estrutura**.

Onde isso aparece no produto

- preenchimento automático
- ingestão de documentos
- CRM e sistemas jurídicos
- pipelines de dados não estruturados

Risco principal

O modelo pode inferir algo que não está explícito, preencher campos com “suposições”. Por isso, é comum exigir evidência textual, marcar campos como “inferido” vs “extraído”, validar antes de persistir.

Geração de texto estruturado

Texto estruturado:

- segue uma estrutura fixa
- é repetível
- é auditável
- pode ser editado e validado
- vira decisão, ação ou contrato

Exemplos:

- relatórios executivos
- diagnósticos técnicos
- propostas comerciais
- dossiês analíticos
- pareceres e planos de ação

Geração de texto estruturado

Errado: “Escreva um relatório completo sobre X”

Isso faz o modelo inventar a estrutura, variar formato a cada execução, misturar fatos, hipóteses e opinião e dificultar revisão e confiança.

LLMs não devem decidir a estrutura.

Estrutura primeiro, texto depois

1. Você define a estrutura
2. O modelo preenche o conteúdo
3. O sistema valida
4. A UI apresenta e permite ajuste

Esse princípio vale para qualquer tipo de documento.

Dados, inferências e limites

Um LLM precisa saber:

1. o que veio dos dados
2. o que é inferência
3. o que não pode ser afirmado

Boa prática:

- marcar hipóteses como hipóteses
- exigir evidência para afirmações fortes
- permitir campos vazios quando não há dado

Schema em Geração de Texto Estruturado

Schema é um contrato entre o sistema, o modelo e o produto. Ele define i) o que deve existir, ii) em que formato, iii) com que tipo de conteúdo e iv) com que limites.

Schema diz como o documento deve ser, não o que o modelo acha bonito escrever.

Schema pode existir como:

- modelo mental
- checklist
- template fixo
- formulário
- contrato de API
- JSON / YAML / Pydantic

Nível 1 de schema: estrutura de seções

O nível mais básico de schema define seções obrigatórias.

Mesmo que uma seção fique vazia, ela existe.

Isso já elimina:

- textos caóticos
- omissões críticas
- estrutura variável

Nível 2 de schema: campos dentro das seções

Agora cada seção tem campos explícitos.

```
{  
  "objetivo": {  
    "descricao": "",  
    "publico_alvo": "",  
    "decisao_suportada": ""  
  }  
}
```

Aqui o modelo não pode escrever um parágrafo solto.

Ele precisa preencher campos.

Nível 3 de schema: tipos de conteúdo

Cada campo tem um tipo semântico, não só técnico.

Exemplos: lista, número, enum, hipótese, evidência, recomendação, etc.

```
{
  "achados": [
    {
      "descricao": "",
      "tipo": "fato | inferencia | risco",
      "evidencias": []
    }
  ]
}
```

Isso força o modelo a classificar o que está dizendo e não tratar opinião como fato.

Separar fato, inferência e recomendação

Recurso poderoso para que modelo se organize. Um fato é uma afirmação verificável e objetiva. Uma inferência é uma conclusão tirada a partir de evidências ou fatos, mas não diretamente observada. Uma recomendação é uma sugestão para uma ação futura.

```
{  
  "analise": {  
    "fatos_observados": [],  
    "inferencias": [],  
    "recomendacoes": []  
  }  
}
```

Schema com níveis de confiança

Outra prática madura: exigir confiança explícita.

```
{  
  "achado": "",  
  "nivel_confianca": "alto | medio | baixo",  
  "motivo_confianca": ""  
}
```

Isso não torna o modelo perfeito, mas torna a incerteza visível. Através disso é possível criar regras, por exemplo: se o conteúdo tiver nível de confiança baixo, será oculto ou explicitamente marcado como baixo (princípio da transparência).

Schema com evidência obrigatória

Para contextos críticos, afirmação sem evidência não entra.

```
{  
  "recomendacao": "",  
  "justificativa": "",  
  "evidencias": [  
    {  
      "fonte": "",  
      "trecho": ""  
    }  
  ]  
}
```

Se não houver evidência o campo fica vazio ou vira hipótese explicitamente marcada. O modelo é forçado a não inventar. Porém, tem o trade-off de que se não tiver informações suficientes o documento retornado será pobre.

Schema orientado a decisão

Alguns documentos são usados para tomada de decisão. Schema pode deixar isso explícito.

```
{
  "decisao_sugerida": "",
  "opcoes_consideradas": [
    {
      "opcao": "",
      "pros": [],
      "contras": []
    }
  ],
  "impacto_esperado": ""
}
```

Onde entram os agentes?

Em documentos estruturados, agentes podem atuar como:

- planejador (define outline)
- pesquisador (coleta evidências)
- redator (gera texto)
- revisor (valida consistência)
- formatador (aplica layout)

Separar papéis aumenta qualidade e controle.

Problemas na Geração de Documentos Estruturados

Cenário comum:

1. Recebo dados
2. Crio um prompt
3. Resultado varia
4. Evidências se perdem
5. Difícil validar e escalar

O problema é que:

- a entrada não é normalizada
- a saída não é contratualizada
- não existe validação entre gerar e publicar

Pipeline Ideal

Visão conceitual do processo

1. Dados brutos
2. Normalização para input
3. Planejamento do documento
4. Geração por seção com schema
5. Validação e revisão
6. Renderização final (documento / UI)

Nota: Cada etapa é detalhada no [material de apoio](#), com exemplos de schema, prompts e validações.

Sistemas Conversacionais

São softwares projetados para interagir com usuários de forma natural, simulando uma conversa humana através de texto ou voz. Eles interpretam intenções, mantêm o contexto do diálogo e aprendem continuamente, sendo aplicados principalmente no atendimento ao cliente, suporte e automação de tarefas. É um pipeline com estado, regras, dados e controle, onde o LLM é um componente, não o sistema inteiro.

Agentes e orquestração

A Orquestração é o processo de gerenciar, coordenar e sequenciar o trabalho de múltiplos agentes (ou de um único agente executando várias tarefas). É o "maestro" do sistema.

Sem orquestração, os agentes ficariam perdidos ou tentariam fazer tudo de uma vez. A orquestração define:

- O Fluxo: Quem começa? Quem revisa o trabalho de quem?
- A Memória: Como as informações de um passo são passadas para o próximo.
- O Controle: Quando o processo deve parar ou pedir ajuda humana.

Como múltiplos agentes trabalham juntos?

Imagine que você quer criar um relatório de mercado sobre a Nvidia. Em um sistema orquestrado, teríamos:

Papel	Agente	Tarefa
Pesquisador	Agente A	Varre a internet em busca das notícias mais recentes e dados financeiros.
Analista	Agente B	Pega os dados brutos e identifica tendências de crescimento ou riscos.
Escritor	Agente C	Transforma a análise em um texto elegante e formatado.
Revisor	Agente D	Checa se os fatos estão corretos e se o tom é profissional.

A Orquestração garante que o Escritor não comece a digitar antes do Pesquisador terminar, e que o Revisor mande o texto de volta para o Escritor caso encontre um erro.

Prós

- **Especialização e Precisão:** É o princípio do "dividir para conquistar". Um agente configurado apenas para "revisar código" será muito mais rigoroso do que um modelo geral tentando programar e revisar ao mesmo tempo. Isso reduz as chamadas "alucinações".
- **Modularidade:** Se você precisar mudar a forma como seu sistema faz buscas na web, você altera apenas o "Agente Pesquisador". O resto da orquestração permanece intacto. Isso facilita a manutenção e atualização do sistema.
- **Escalabilidade de Raciocínio:** Problemas complexos que excedem a janela de contexto ou a capacidade lógica de uma única resposta podem ser quebrados em 10 etapas menores. Cada agente resolve uma parte, mantendo o foco total naquela subtarefa.
- **Capacidade de Autocorreção:** Na orquestração, você pode colocar um agente para vigiar o outro. Se o "Agente A" entregar um resultado medíocre, o "Agente B" (Crítico) pode rejeitar o output e mandar o Agente A refazer até que esteja perfeito.

Contras

- **Custo Elevado:** Cada agente faz suas próprias chamadas de API. Um fluxo com 5 agentes trocando mensagens entre si consome muito mais tokens do que uma única pergunta direta. O custo pode escalar rapidamente se o fluxo não for otimizado.
- **Latência (Demora):** Mais agentes significam mais tempo de processamento. Um sistema multi-agente raramente é "em tempo real". A resposta pode levar segundos ou até minutos, dependendo da complexidade da orquestração.
- **Propagação de Erros:** Se o primeiro agente da fila cometer um erro sutil e a orquestração não tiver um filtro de qualidade eficiente, esse erro será levado adiante e amplificado por todos os agentes subsequentes.
- **Dificuldade de Debugging:** Quando algo dá errado, pode ser difícil identificar onde o fluxo quebrou. Foi a instrução do maestro? Foi o Agente 2 que interpretou mal o Agente 1? Rastrear a "conversa" entre as máquinas exige ferramentas de monitoramento avançadas (como LangSmith ou Arize).

LLMs como camada de decisão (com cuidado)

Para implementar isso com responsabilidade, usam-se padrões de design:

- **Padrão Planejador-Executor:** Uma LLM cria o plano de ação, e outra (ou um código rígido) revisa se o plano faz sentido antes de qualquer execução.
- **Decisão Baseada em Esquema (Structured Output):** Forçar a LLM a responder em JSON ou formatos pré-definidos. Isso impede que ela "se perca na conversa" e garante que a decisão caia em uma categoria que seu sistema entende.
- **Human-in-the-loop (HITL):** Para decisões de alto impacto (financeiras, médicas, deleção de dados), a LLM propõe a decisão, mas um humano deve clicar em "Aprovar".

O que não é um bom caso de uso

1. Operações Matemáticas e Lógicas Determinísticas
2. Processos de Baixa Latência e Altíssima Escala
3. Dados Altamente Dinâmicos sem RAG
4. Decisões Éticas ou Jurídicas de "Caixa Preta"
5. Substituição de Regras de Negócio Rígidas

Fuja da LLM se...	Use em vez disso...
O resultado precisa ser 100% exato (matemática).	Calculadoras / Código.
O custo por transação precisa ser quase zero .	Heurísticas ou ML Simples.
A resposta precisa ser instantânea (<100ms).	Modelos locais ou Regras.
A regra de decisão é imutável e clara .	Lógica de programação tradicional.

Engenharia de Prompt



O que é prompt engineering — e o que não é

Prompt engineering é a prática de estruturar instruções para um modelo de linguagem de forma intencional, com o objetivo de induzir comportamentos previsíveis, alinhados ao contexto de negócio, às regras do produto e às expectativas do usuário. Ela envolve decisões conscientes sobre papel do sistema, escopo de atuação, limites, formato de resposta e critérios de sucesso.

Prompt engineering não é:

- "Magia" ou "Feitiço": Não existem palavras mágicas que funcionam sempre.
- Programação Tradicional: Diferente do código (Python, C++), o prompt é baseado em linguagem natural e é inerentemente não-determinístico (o mesmo prompt pode gerar resultados levemente diferentes).
- Uma Solução para Tudo: Se o modelo não possui o conhecimento (falta de dados) ou se a tarefa exige lógica matemática complexa que ultrapassa sua capacidade, nenhum prompt "perfeito" resolverá o problema.

Estrutura de um bom prompt: papel

Define a autoridade e o tom da resposta. Em vez de uma resposta "padrão", você evoca um conjunto específico de conhecimentos e comportamentos.

Exemplo: "Atue como um Especialista em Segurança da Informação com foco em conformidade bancária."

Estrutura de um bom prompt: contexto

Fornece as informações de base necessárias para que a IA entenda o cenário. Sem contexto, a IA alucina ou dá respostas óbvias demais.

Exemplo: "Estamos desenvolvendo um chatbot para a B3 que atenderá emissores de balcão sobre regulamentações da CVM."

Estrutura de um bom prompt: tarefa

É o verbo de ação. Deve ser clara, direta e, preferencialmente, única por bloco de prompt para evitar diluição de atenção.

Exemplo: "Elabore um resumo executivo dos principais pontos do documento 'Instrução CVM 358' que impactam o dia a dia desse atendente."

Estrutura de um bom prompt: restrições

Onde você define as "bordas" do campo. Isso evita que a IA escreva demais, use termos proibidos ou entregue em um formato indesejado.

Exemplo: "Não use termos técnicos sem explicá-los. O texto deve ter no máximo 300 palavras e ser entregue em tópicos (bullet points)."

Exemplo de Prompt Estruturado

[Papel] Atue como uma pesquisadora acadêmica sênior especializada em IA e Ética.

[Contexto] Estou realizando uma Revisão Sistemática da Literatura sobre viés social em modelos de linguagem no Brasil.

[Tarefa] Analise o parágrafo abaixo e identifique possíveis lacunas na metodologia de mitigação de viés proposta.

[Restrições] Foque apenas em aspectos técnicos de métricas de fairness. Não responda com introduções genéricas; vá direto aos pontos críticos em formato de tabela.

Zero-Shot (Zero Exemplos)

É quando você pede para a IA realizar uma tarefa sem fornecer nenhum exemplo prévio. Você confia inteiramente no conhecimento pré-existente do modelo.

Quando usar: Tarefas simples, rotineiras ou quando o modelo já é muito robusto no assunto.

Exemplo: "Classifique o sentimento deste comentário: 'O sistema de busca da B3 é muito intuitivo'."

Few-Shot (Poucos Exemplos)

Você fornece alguns exemplos de entrada e saída antes de pedir a execução da tarefa final. Isso "calibra" o modelo para o formato, tom e nível de detalhe que você espera.

Quando usar: Tarefas complexas, classificação com rótulos específicos ou quando você quer um formato de saída muito rígido.

Exemplo:

"O mercado fechou em alta." -> Sentimento: Positivo.

"Houve queda no volume de negociação." -> Sentimento: Negativo.

"O Ibovespa operou em estabilidade." -> Sentimento: [Complete aqui]

Exemplos Negativos (O que NÃO fazer)

Muitas vezes, dizer o que você quer não é suficiente; você precisa mostrar o que não serve. Isso é extremamente útil para evitar erros comuns ou comportamentos indesejados (como o modelo ser prolixo ou amigável demais em um contexto técnico).

Quando usar: Para reduzir "alucinações", evitar ambiguidades ou garantir que a IA não use termos proibidos.

Exemplo:

Exemplo Positivo: "O lucro cresceu 10%." -> Resumo: Crescimento de 10%.

Exemplo Negativo (Não faça assim): "O lucro cresceu 10%." -> Resumo: A empresa teve um desempenho excelente e os investidores estão felizes. (Motivo: Não adicione opiniões subjetivas).